

Kaczmarz Linear Attention

Jiaxuan Zou

School of Mathematics and Statistics
Xi'an Jiaotong University
Xi'an, China

Ruifeng Ren

Gaoling School of Artificial Intelligence
Renmin University of China
Beijing, China

Yong Liu*

Gaoling School of Artificial Intelligence
Renmin University of China
Beijing, China
liuyonggsai@ruc.edu.cn

Abstract

Long-context language modeling remains central to modern sequence modeling, but the quadratic cost of Transformer attention makes scaling computationally prohibitive. Linear recurrent models address this bottleneck by compressing the context into a fixed-size state, making the rule that forgets, writes, and edits information a central design problem. To address state maintenance, Gated DeltaNet (GDN) combines gated state decay with delta-rule residual writes, using a learnable coefficient to balance forgetting and update magnitude. However, this coefficient is learned empirically rather than derived from the underlying objective, which can lead to suboptimal update magnitudes. We revisit the online-regression objective underlying GDN and, inspired by the Kaczmarz projection method, derive the key-norm-normalized dynamic step size $\beta_t = \eta_t / (\|k_t\|_2^2 + \epsilon)$ for residual updates. We propose **Kaczmarz Linear Attention** (KLA), a one-scalar modification of GDN that preserves the state shape, gates, linear recurrence, and chunkwise parallel algorithm. At the 0.4B scale with a 1B-token budget, KLA achieves the lowest validation perplexity among evaluated linear-time baselines (8.09 vs. 8.50 for GDN) and remains stable up to 65K tokens. On controlled tasks, KLA reaches 100% on single-needle-in-a-haystack (S-NIAH) retrieval, improves $8\times$ multi-query associative recall (MQAR) by 7.03 points over GDN, and delivers $2.1\times$ higher decode throughput at 32K context. These results suggest that the key-norm-normalized Kaczmarz coefficient is a first-order design axis for delta-rule sequence models: it improves accuracy, extrapolation, and decoding efficiency without changing the recurrent state or hardware kernel. An anonymized implementation is included in the supplementary material and available at <https://github.com/anonymous-kla-review/kla-anonymous>.

1 Introduction

Softmax attention gives Transformers strong associative recall, but its token-token attention matrix scales quadratically with sequence length, increasing long-context prefill latency, activation memory, and train-short/test-long evaluation cost [24, 29]. IO-aware kernels and sequence-parallel implementations reduce constant factors, yet they do not remove this asymptotic cost [38, 2, 8, 6, 4, 22].

Linear-time sequence models seek to alleviate this bottleneck by replacing token-token attention with a fixed-size recurrent state. As all past-token information is compressed into bounded memory,

*Corresponding author.

the state-update rule—which determines what the model writes, overwrites, preserves, and later retrieves—becomes the central design choice. Different lines of work propose different update rules based on distinct design insights. Linear attention and fast-weight models use additive key-value writes [16, 27]. Gated recurrent attention adds data-dependent decay [35, 23, 43, 26]. Selective state-space or hybrid models scale related recurrence mechanisms to large language models [11, 7, 9, 19]. Although these models reduce quadratic attention to linear-time recurrence, their writes do not explicitly account for the key-value associations already stored in the state. When similar keys recur, repeated writes can redundantly accumulate related information instead of editing the existing association.

Delta-rule models address this redundancy by writing the residual between the target value and the value predicted by the current state. DeltaNet therefore overwrites a specific key direction instead of only adding or decaying information [44]. To control the residual-write magnitude more finely, Gated DeltaNet (GDN) introduces a learned scalar that balances forgetting and updating and improves long-context retrieval [45]. However, this scalar is primarily an empirical design choice rather than a theoretically specified step size. An inappropriate coefficient can over-correct large-norm keys, under-correct small-norm keys, and leave the write scale mismatched to the current key. Because the GDN update admits an online-regression view, the coefficient can instead be derived from the per-token optimization problem.

KLA resolves this coefficient problem by treating each delta-rule write as a Kaczmarz projection. The key insight is simple: a key-value write imposes one linear constraint, $S_t^\top k_t = v_t$, and the closest state satisfying that constraint is reached with the Kaczmarz step size $\beta_t = \eta_t / (\|k_t\|_2^2 + \varepsilon)$ [15]. This self-normalized projection turns the gated delta rule into an exact per-token update in the unrelaxed case while preserving the GDN state shape, gates, linear recurrence, and chunkwise parallel kernel. Thus KLA changes the coefficient, not the architecture.

We make four contributions:

- **Self-normalized delta rule.** We derive KLA from the Kaczmarz projection and show that its coefficient is simultaneously an orthogonal projection step, an exact line-search minimizer, and a normalized gradient step for the per-token loss.
- **Drop-in recurrent algorithm.** We show that KLA preserves the GDN recurrence and chunkwise solver; the kernel changes only through the diagonal coefficient matrix $B = \text{Diag}(\beta_1, \dots, \beta_C)$.
- **Empirical gains.** At the 0.4B scale with a 1B-token budget, KLA obtains the lowest validation perplexity among evaluated linear-time baselines and improves synthetic retrieval and state-tracking, including single-needle-in-a-haystack (S-NIAH), multi-query associative recall (MQAR), and Stack.
- **Preserved efficiency.** KLA keeps $O(n)$ time and memory complexity, matches GDN prefill latency, and improves decode throughput without a new state structure or hardware kernel.

Table 1 summarizes the main language-modeling result.

	KLA	DeltaNet	GLA	GDN	Longhorn	Mamba2
Validation PPL ↓	8.09	8.26	8.42	8.50	8.79	9.41

Table 1: Pretraining validation perplexity (PPL) at the 0.4B scale with a 1B-token budget on SlimPajama-672B [33]. All models use the same tokenizer, optimizer, and 4K training context.

2 Background and Problem Analysis

Linear-time sequence models replace the token-token attention matrix with a recurrent state $S_t \in \mathbb{R}^{d_k \times d_v}$, so the main design choice is the write rule applied to this state. Linear attention writes each key-value pair additively, $S_t = S_{t-1} + k_t v_t^\top$, and a query reads $o_t = S_t^\top q_t$ [16]. Gated Linear Attention (GLA) adds data-dependent forgetting, $S_t = \text{Diag}(\alpha_t) S_{t-1} + k_t v_t^\top$, so earlier content can decay before the next write [43]. In the scalar-gated notation used below, this forget-then-write pattern is

$$\tilde{S}_{t-1} = \alpha_t S_{t-1}, \quad S_t = \tilde{S}_{t-1} + k_t v_t^\top.$$

These recurrent forms are efficient; the delta-rule family changes the write itself.

Delta-rule models make the state content-editable by writing the residual between the target value and the current state prediction. In the scalar-gated GDN template [45], the residual is measured and written back along the current key direction:

$$S_t = \tilde{S}_{t-1} + \beta_t k_t e_t^\top = (I - \beta_t k_t k_t^\top) \tilde{S}_{t-1} + \beta_t k_t v_t^\top, \quad e_t = v_t - \tilde{S}_{t-1}^\top k_t.$$

The rank-one correction edits the state in the active key direction, and β_t determines the step length of this edit. After decay, token t can be viewed through a data-fit loss and its regularized online-regression form:

$$\mathcal{L}_t(S) = \frac{1}{2} \|S^\top k_t - v_t\|_2^2, \quad \mathcal{J}_t(S; \lambda_t) = \mathcal{L}_t(S) + \frac{\lambda_t}{2} \|S - \tilde{S}_{t-1}\|_F^2.$$

Here $\mathcal{L}_t$ fits the current key-value association, while the regularization term keeps the new state close to the decayed memory. The residual direction $k_t e_t^\top$ is the negative gradient direction of $\mathcal{L}_t$ at $\tilde{S}_{t-1}$, and the scalar coefficient controls how strongly this residual is written. In the GDN template, this coefficient is supplied by a learned gate, $\beta_t = \eta_t$; equivalently, the update magnitude is learned empirically rather than derived from the regularized objective. The next section derives this coefficient from the per-token optimization problem itself.

3 Kaczmarz Linear Attention

3.1 Kaczmarz coefficient from constrained optimization

The regularized view above isolates the residual direction from the scalar that controls its magnitude. We now derive the scalar by treating a key-value write as a single linear constraint on the recurrent state. The Kaczmarz method is a classical row-action method for solving linear systems by repeatedly projecting the current iterate onto one constraint hyperplane at a time [15]. For token t , we choose the state closest to the decayed state $\tilde{S}_{t-1}$ that satisfies the current key-value constraint:

$$\min_{S \in \mathbb{R}^{d_k \times d_v}} \frac{1}{2} \|S - \tilde{S}_{t-1}\|_F^2 \quad \text{subject to} \quad S^\top k_t = v_t. \quad (1)$$

This problem asks for the minimum-norm state change that makes key k_t read out value v_t . The constraint set $\mathcal{A}_t = \{S : S^\top k_t = v_t\}$ is an affine subspace with normal direction k_t , so the optimal correction has the form $S_t = \tilde{S}_{t-1} + k_t c^\top$ for some $c \in \mathbb{R}^{d_v}$. Substituting this form into the constraint gives $\tilde{S}_{t-1}^\top k_t + c \|k_t\|_2^2 = v_t$, hence $c = e_t / \|k_t\|_2^2$, and the exact projection is

$$S_t = \tilde{S}_{t-1} + \frac{1}{\|k_t\|_2^2} k_t e_t^\top. \quad (2)$$

Thus Kaczmarz writes the residual e_t with a step length normalized by the key energy. The practical KLA update adds a learned relaxation gate and a numerical stabilizer,

$$S_t = \tilde{S}_{t-1} + \beta_t k_t e_t^\top, \quad \beta_t = \frac{\eta_t}{\|k_t\|_2^2 + \varepsilon}. \quad (3)$$

When $\eta_t = 1$ and $\varepsilon = 0$, (3) reduces to the exact Kaczmarz projection in (2). The relaxed update also has an online-regression interpretation:

Proposition 1 (Exact proximal form). *For any $\eta_t \in (0, 1]$ and $\varepsilon \geq 0$, define $\mu_t = \eta_t / ((1 - \eta_t) \|k_t\|_2^2 + \varepsilon)$. The relaxed update in (3) is the exact minimizer of*

$$\min_S \frac{1}{2} \|S - \tilde{S}_{t-1}\|_F^2 + \frac{\mu_t}{2} \|S^\top k_t - v_t\|_2^2.$$

Thus, the practical coefficient can be read either as a relaxed Kaczmarz step or as the closed-form solution of a proximal local regression problem.

Connection to GDN. To make the coefficient-level comparison explicit, return to the data-fit loss $\mathcal{L}_t$ defined in section 2. GDN and KLA use the same decayed state $\tilde{S}_{t-1}$, residual e_t , and rank-one write direction $k_t e_t^\top$; they differ only in the scalar multiplying that write. Along the one-parameter path $S(\tau) = \tilde{S}_{t-1} + \tau k_t e_t^\top$, substituting into $\mathcal{L}_t$ gives

$$\mathcal{L}_t(S(\tau)) = \frac{1}{2} (1 - \tau \|k_t\|_2^2)^2 \|e_t\|_2^2. \quad (4)$$

The exact line-search coefficient is therefore $\tau^* = 1/\|k_t\|_2^2$. GDN uses $\beta_t = \eta_t$, so it matches this step only when the learned scalar happens to equal $1/\|k_t\|_2^2$ for the current token. In contrast, KLA uses $\beta_t = \eta_t/(\|k_t\|_2^2 + \varepsilon)$ and recovers the exact minimizer when $\eta_t = 1$ and $\varepsilon = 0$. The improvement is not a new write direction; it is a key-energy correction to the GDN step length.

Table 2 places this coefficient-level change alongside representative recurrent layers. The comparison separates the local objective from the update rule while keeping the recurrent state fixed.

Method	Local objective $\mathcal{L}_t$	Update rule
Linear attention	$-\langle S_{t-1}^\top k_t, v_t \rangle$	$S_t = S_{t-1} + k_t v_t^\top$
RetNet / Mamba2	$-\beta_t \langle S_{t-1}^\top k_t, v_t \rangle + \frac{1}{2} \ \sqrt{1 - \alpha_t} S_{t-1}\ _F^2$	$S_t = \alpha_t S_{t-1} + \beta_t k_t v_t^\top$
GLA	$-\langle S_{t-1}^\top k_t, v_t \rangle + \frac{1}{2} \ \sqrt{\text{Diag}(1 - \alpha_t)} S_{t-1}\ _F^2$	$S_t = \text{Diag}(\alpha_t) S_{t-1} + k_t v_t^\top$
Longhorn	$\frac{\beta_t}{2} \ S_{t-1}^\top k_t - v_t\ _2^2$	$S_t = (I - \rho_t k_t k_t^\top) S_{t-1} + \rho_t k_t v_t^\top$
DeltaNet	$\frac{\beta_t}{2} \ S_{t-1}^\top k_t - v_t\ _2^2$	$S_t = (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top$
GDN	$\frac{\beta_t}{2} \ S_{t-1}^\top k_t - v_t\ _2^2$	$S_t = (I - \beta_t k_t k_t^\top) \tilde{S}_{t-1} + \beta_t k_t v_t^\top$
KLA	$\frac{1}{2} \ \tilde{S}_{t-1}^\top k_t - v_t\ _2^2$	$S_t = \tilde{S}_{t-1} + \frac{\eta_t}{\ k_t\ _2^2 + \varepsilon} k_t (v_t - \tilde{S}_{t-1}^\top k_t)^\top$

Table 2: Online-learning view of recurrent updates. KLA is the only model in the table that exactly minimizes the per-token loss in the unrelaxed case ($\eta_t = 1$, $\varepsilon = 0$).

3.2 Why the Kaczmarz coefficient is natural

The constrained optimization view gives a candidate coefficient, and the following result explains why this coefficient is natural. The same scalar simultaneously satisfies three desiderata that are usually treated separately in online learning: it is an orthogonal projection, an exact line-search minimizer, and a normalized gradient step. This unified characterization justifies why the key-norm term should appear in the delta-rule coefficient rather than in an auxiliary heuristic.

Theorem 1 (Projection, line search, and normalized SGD). *Assume $\eta_t = 1$ and $\varepsilon = 0$. Let $\tilde{S}_{t-1} = \alpha_t S_{t-1}$, $e_t = v_t - \tilde{S}_{t-1}^\top k_t$, and $\mathcal{A}_t = \{S : S^\top k_t = v_t\}$. The update*

$$S_t = \tilde{S}_{t-1} + \frac{1}{\|k_t\|_2^2} k_t e_t^\top$$

satisfies $S_t^\top k_t = v_t$. Moreover, the same update is simultaneously:

- (i) *the orthogonal projection of $\tilde{S}_{t-1}$ onto $\mathcal{A}_t$ in Frobenius norm;*
- (ii) *the exact minimizer of $\mathcal{L}_t(S)$ along the direction $\tilde{S}_{t-1} + \tau k_t e_t^\top$; and*
- (iii) *a normalized SGD update on $\mathcal{L}_t$ with coefficient $1/\|k_t\|_2^2$.*

Remark 1. *By the projection theorem for Hilbert spaces, S_t is the unique element of $\mathcal{A}_t$ closest to $\tilde{S}_{t-1}$. The Kaczmarz update makes the minimum-norm change to the state while exactly satisfying the constraint.*

Proposition 2 (Residual contraction). *Let $e_t = v_t - \tilde{S}_{t-1}^\top k_t$ and $e_t^+ = v_t - S_t^\top k_t$. Then*

$$e_t^+ = \left(1 - \eta_t \frac{\|k_t\|_2^2}{\|k_t\|_2^2 + \varepsilon}\right) e_t. \quad (5)$$

For $0 < \eta_t \leq 1$, the residual contracts and the per-token loss decreases monotonically: $\mathcal{L}_t(S_t) \leq \mathcal{L}_t(\tilde{S}_{t-1})$.

Equation (5) gives the stability condition for the relaxed update. The contraction factor is bounded by the gate η_t and regularized by ε , so the update cannot amplify the current residual. The Kaczmarz coefficient depends on $\tilde{S}_{t-1}$ and k_t , not on the form of the decay operator; therefore KLA and KDA [36] are orthogonal improvements. Proofs are in section A.

3.3 Tokenwise implementation

At the token level, KLA keeps the same decay, residual computation, and rank-one write as GDN. The implementation changes only the coefficient line:

GDN <pre> Sbar = alpha * S err = v - Sbar^T @ k beta = eta S = Sbar + beta * outer(k, err) </pre>	KLA <pre> Sbar = alpha * S err = v - Sbar^T @ k beta = eta / (norm(k)**2 + eps) S = Sbar + beta * outer(k, err) </pre>
--	---

The left panel of algorithm 1 gives the per-token update, and the right panel gives the matching chunkwise solver. As in GDN, η_t is a sigmoid-activated linear projection of the input token. The query q_t is ℓ_2 -normalized before the readout $o_t = S_t^T q_t / \|q_t\|_2$.

Algorithm 1 Tokenwise and chunkwise KLA computation

Tokenwise update (one head)	Chunkwise kernel (chunk length C)
Require: $S_{t-1}, \alpha_t, k_t, v_t, q_t, \eta_t, \varepsilon$ 1: $\tilde{S}_{t-1} \leftarrow \alpha_t S_{t-1}$ 2: $e_t \leftarrow v_t - \tilde{S}_{t-1}^T k_t$ 3: $\beta_t \leftarrow \eta_t / (\ k_t\ _2^2 + \varepsilon)$ 4: $S_t \leftarrow \tilde{S}_{t-1} + \beta_t k_t e_t^T$ 5: $o_t \leftarrow S_t^T q_t / \ q_t\ _2$ 6: return o_t, S_t	Require: $S_0, K, V, Q, \{\alpha_i\}, \{\eta_i\}, \varepsilon$ 1: for $i = 1$ to C do 2: $\gamma_i \leftarrow \alpha_i \gamma_{i-1}; \beta_i \leftarrow \eta_i / (\ k_i\ _2^2 + \varepsilon)$ 3: end for 4: Form $B = \text{Diag}(\beta_1, \dots, \beta_C), D_\gamma, A, A^-$ 5: Solve $(I + B(A^- \odot K K^T))U = B(V - D_\gamma K S_0)$ 6: $O \leftarrow D_\gamma Q S_0 + (A \odot Q K^T)U$ 7: $S_{\text{out}} \leftarrow \gamma_C S_0 + K^T \text{Diag}(\gamma_C / \gamma_1, \dots, 1)U$ 8: return O, S_{out}

4 Chunkwise Parallelism

KLA inherits the GDN chunkwise-parallel training algorithm because the Kaczmarz coefficient changes only the scalar multiplying each residual write. With $R_t = S_t^T \in \mathbb{R}^{d_v \times d_k}$, the update in (3) can be written as

$$R_t = \alpha_t R_{t-1} (I - \beta_t k_t k_t^T) + \beta_t v_t k_t^T, \quad \beta_t = \frac{\eta_t}{\|k_t\|_2^2 + \varepsilon}. \quad (6)$$

This is the GDN recurrence with a different diagonal coefficient. A comparison with representative recurrent and chunkwise-parallel forms is deferred to table 6; the within-chunk triangular system is derived in section B.

For a chunk of length C , define stacked matrices $K, V, Q \in \mathbb{R}^{C \times d}$, auxiliary matrix $U \in \mathbb{R}^{C \times d_v}$ with rows u_j^T , $B = \text{Diag}(\beta_1, \dots, \beta_C)$, $D_\gamma = \text{Diag}(\gamma_1, \dots, \gamma_C)$, and $\gamma_i = \prod_{r=1}^i \alpha_r$. Let $A_{ij} = \gamma_i / \gamma_j$ for $j \leq i$ and zero otherwise, and let A^- be the strict lower-triangular part of A . These are the same objects used by the GDN solver; only B is formed from the Kaczmarz coefficient.

Proposition 3 (Chunkwise solve). *The auxiliary vectors U satisfy the lower-triangular linear system*

$$(I + B(A^- \odot K K^T))U = B(V - D_\gamma K S_0). \quad (7)$$

The left-hand matrix has unit diagonal and can be solved by forward substitution or the GDN chunkwise kernel.

Proposition 4 (Chunk outputs and outgoing state). *The chunk outputs and final state are*

$$O = D_\gamma Q S_0 + (A \odot Q K^T)U, \quad (8)$$

$$S_{\text{out}} = \gamma_C S_0 + K^T \text{Diag}(\gamma_C / \gamma_1, \dots, 1)U. \quad (9)$$

Propositions 3 and 4 are proved in section B. They show that KLA uses the same lower-triangular solve and the same outgoing-state formula as GDN once B is populated with $\eta_i/(\|k_i\|_2^2 + \varepsilon)$. Algorithm 1 summarizes the tokenwise and chunkwise computations side by side, avoiding two separate algorithm floats.

5 Experiments

5.1 Setup

Pretraining. We evaluate whether the Kaczmarz coefficient improves language modeling under fixed compute and fixed architecture. All models have 0.4B parameters and train on 1B tokens from SlimPajama-672B [33] with a 4K context, following the GDN pipeline [45] and standard language-model scaling practice [14, 37]. All models use the same tokenizer, AdamW optimizer, and cosine schedule.

Baselines. We compare KLA with GDN [45], DeltaNet [44], GLA [43], Longhorn [21], and Mamba2 [7] for pretraining perplexity. For long-context, synthetic, and efficiency evaluations, we use KLA, GDN [45], and Mamba2 [7] as a consistent three-way comparison. All baselines use the same architecture, parameter count, and training setup.

Evaluation. We evaluate three properties: language modeling, controlled state use, and runtime efficiency. Metrics include validation perplexity from 1K to 65K context without long-context fine-tuning, accuracy on four synthetic tasks, and prefill plus decode metrics from the FLA library [42]. The synthetic tasks are MQAR (multi-query associative recall), S-NIAH (single needle-in-a-haystack), Palindrome (structured reversal), and Stack (push-pop state tracking).

5.2 Language Modeling

The first question is whether replacing GDN’s learned scalar with the Kaczmarz coefficient lowers perplexity under the same model size, data, optimizer, and context length.

Pretraining perplexity. We hypothesize that a coefficient that solves the local key-value regression problem should improve next-token prediction under the same compute budget. Table 1 reports that KLA reaches 8.09 PPL, outperforming DeltaNet (8.26), GLA (8.42), GDN (8.50), Longhorn (8.79), and Mamba2 (9.41). The 0.41 PPL improvement over GDN supports the hypothesis that the learned scalar in GDN leaves measurable loss on the table. Figure 1 shows the same effect in validation perplexity during pretraining: KLA converges to a lower final validation PPL than the baselines.

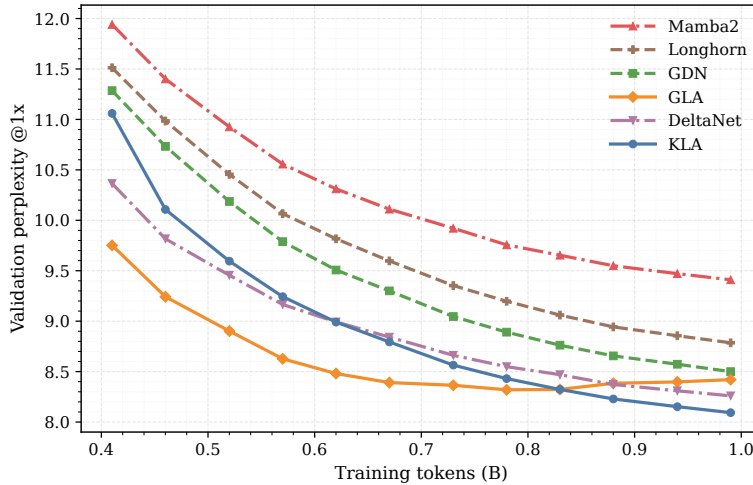


Figure 1: Validation perplexity curves during pretraining. KLA converges to a lower final validation PPL than all baselines.

Long-context perplexity. We also test whether the coefficient correction remains useful outside the 4K training context. Table 3 reports validation perplexity from 1K to 65K tokens without long-context fine-tuning. KLA achieves the lowest perplexity at every tested length, with a 3.7–4.1 PPL advantage over GDN. This persistent gap supports the interpretation that key-norm-normalized writes reduce state corruption beyond the training length, rather than only improving optimization at 4K tokens. The tabulated values make the per-length gap explicit without relying on a trend plot.

Model	1K	2K	4K	8K	16K	32K	65K
KLA	39.1	39.8	40.0	43.6	45.3	44.5	43.6
GDN	42.9	43.5	43.8	47.4	49.4	48.4	47.5
Mamba2	49.9	50.9	52.2	56.4	58.7	57.3	56.7

Table 3: Long-context validation perplexity ($\downarrow$) by context length, measured in tokens. KLA achieves the lowest perplexity at every length without long-context fine-tuning.

5.3 Synthetic Tasks: Controlled State Use

We use four synthetic tasks to isolate how each model writes to and reads from its recurrent state. All tasks follow the GDN configurations [45]: MQAR is trained at length 256 and evaluated up to $8\times$ length, S-NIAH is evaluated from 1K to 8K context, and Palindrome and Stack are evaluated by sequence length. Table 4 reports length extrapolation, and fig. 2 reports training convergence.

Palindrome. Palindrome requires the model to reproduce a random token sequence in reverse order, testing ordered sequence storage rather than key-value association:

Input	<i>O</i>	<i>G</i>	<i>R</i>	<i>S</i>	<i>U</i>	<i>N</i>	<i>E</i>	$\langle \text{sep} \rangle$	<i>E</i>	<i>N</i>	<i>U</i>	<i>S</i>	<i>R</i>	<i>G</i>	<i>O</i>
Output	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	<i>N</i>	<i>U</i>	<i>S</i>	<i>R</i>	<i>G</i>	ϕ

Multi-Query Associative Recall (MQAR). MQAR tests whether a model can retrieve values associated with multiple query keys appearing at different positions in the context. For example, after observing pairs such as $B \mapsto 0$ and $G \mapsto 5$, the model must output the corresponding values when queried:

Input	<i>A</i>	1	<i>C</i>	3	<i>B</i>	0	<i>M</i>	8	<i>G</i>	5	<i>E</i>	4	$\langle \text{sep} \rangle$	<i>B</i>	<i>G</i>
Output	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	0	5

S-NIAH. Single-needle-in-a-haystack evaluates whether the model can write and retrieve one salient key-value association among long distractor contexts.

Stack. Stack simulates LIFO push-pop state tracking. The input contains push operations that insert elements into one of several stacks and pop operations that require the model to output the most recently pushed element for the queried stack. The task stresses repeated local state edits rather than a single retrieval.

5.4 Efficiency

Efficiency tests whether the Kaczmarz coefficient preserves the runtime profile of the GDN chunkwise algorithm. KLA should leave prefill unchanged because it uses the same chunkwise triangular solve with a different diagonal coefficient matrix.

Prefill latency. KLA matches GDN prefill latency. Figure 3a shows that, at 131K context, prefill takes 1220 ms for KLA, 1227 ms for GDN, and 1936 ms for Mamba2. Computing $\|k_t\|_2^2$ and substituting the Kaczmarz coefficient therefore do not change the dominant chunkwise kernel cost.

Decode throughput. KLA improves decode throughput at long context. At 32K context, KLA reaches 3.54 tokens/s, which is $2.1\times$ higher than GDN (1.65 tokens/s) and $1.7\times$ higher than Mamba2 (2.10 tokens/s). Figure 3b–c reports throughput and time per output token across context lengths, following standard decoding-efficiency metrics [30, 1]. The efficiency result supports the drop-in claim: KLA changes the write coefficient but not the state layout, recurrent interface, or hardware kernel.

MQAR validation accuracy (%)				
Model	256	512	1024	2048
GDN	98.74	98.36	94.31	<u>66.81</u>
KLA	<u>99.00</u>	<u>98.56</u>	<u>95.64</u>	73.84
Mamba2	99.08	99.02	96.54	64.29

S-NIAH exact-match accuracy (%)				
Model	1K	2K	4K	8K
GDN	<u>98.85</u>	<u>98.25</u>	<u>98.05</u>	<u>98.50</u>
KLA	100.00	100.00	100.00	100.00
Mamba2	0.10	0.00	0.00	0.05

Palindrome accuracy (%)				
Model	256	512	1024	2048
GDN	<u>99.94</u>	99.81	<u>98.24</u>	80.74
KLA	99.96	<u>99.50</u>	98.33	<u>79.96</u>
Mamba2	3.20	17.92	88.22	0.86

Stack accuracy (%)				
Model	256	512	1024	2048
GDN	99.93	99.73	98.94	<u>95.64</u>
KLA	<u>99.97</u>	99.96	<u>99.25</u>	96.94
Mamba2	99.98	<u>99.94</u>	99.54	93.82

Table 4: Length extrapolation results for synthetic tasks. All values are accuracies (%). Bold and underline mark the best and second-best values in each column. MQAR is trained at length 256; S-NIAH is evaluated from 1K to 8K context; Palindrome and Stack are evaluated by sequence length.

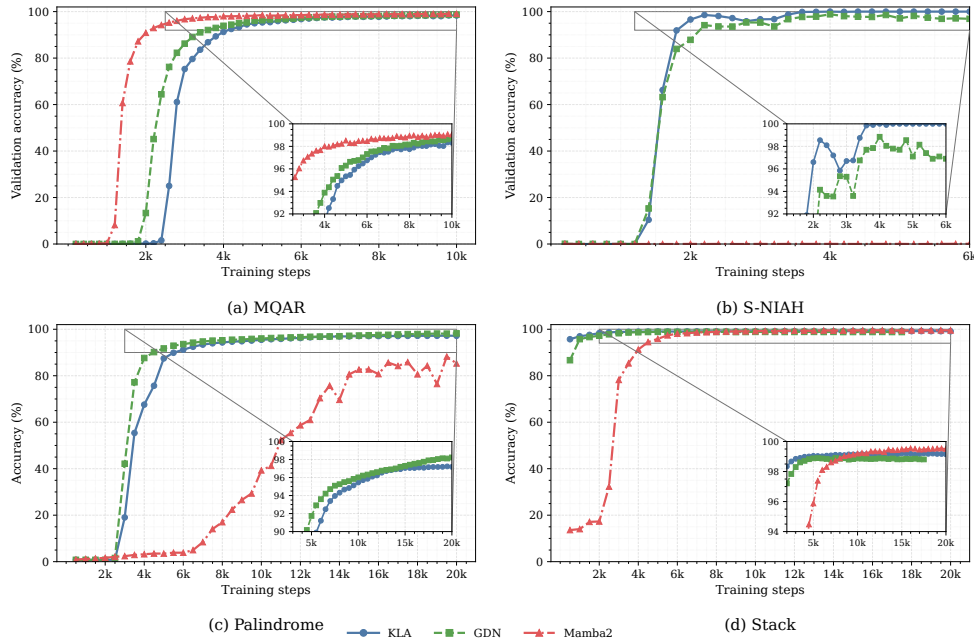


Figure 2: Synthetic task training convergence. Table 4 reports length extrapolation. Insets zoom into the high-accuracy regime for each task.

5.5 Ablation Studies

We ablate three design choices: the Kacmarz normalization, the gating structure, and the value-state expansion ratio. Each variant is evaluated on MQAR at the training length and at $8\times$ extrapolation, and on a 100M-token SlimPajama pretraining run for language-modeling stability. Detailed hyperparameters and variant definitions are in section D.

Normalization strategy. We compare the default coefficient $\beta_t = \eta_t / (\|k_t\|_2^2 + \epsilon)$ with no normalization, key-norm-only normalization, and an unconstrained learned scalar. Table 5 shows that the full Kacmarz coefficient gives the best stability trade-off: normalization is required for MQAR, while the learned scalar and key-norm-only variants do not match the pretraining stability of the default update.

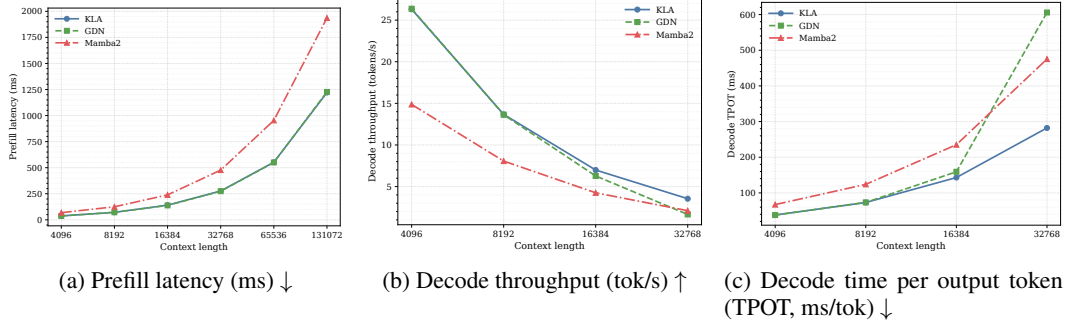


Figure 3: Efficiency comparison with batch size 1. KLA and GDN have nearly identical prefill profiles; KLA achieves $2.1\times$ higher decode throughput than GDN at 32K context.

Gating mechanism. We compare a single-gate variant, the dual-gate GDN reference, and an independent-gate GLA reference. The results test whether correcting the write coefficient is sufficient without the GDN-style gating structure.

State expansion. We vary the value expansion ratio $v_{\text{expand}} \in \{2, 4, 8, 16\}$ to test whether additional state capacity replaces coefficient correctness. Larger expansion can improve synthetic memorization, but it does not remove the language-modeling stability trade-off.

Group	Variant	Test Acc. (%)	$8\times$ Acc. (%)	Val. PPL @ 100M ↓
Normalization	KLA (Default)	98.39	73.84	753.65
Normalization	w/o Normalization	0.03	0.01	—
Normalization	Key-norm only ($1/\ k_t\ _2^2$)	97.81	77.68	3809.71
Normalization	Learned Scalar	97.77	74.46	22968.87
Gating	Single Gate	0.01	0.03	2104.61
Gating	Dual Gate (GDN reference)	98.72	66.81	747.24
Gating	Independent Gate (GLA)	0.03	0.03	356.01
Expansion	$v_{\text{expand}} = 2$	98.09	71.99	762.33
Expansion	$v_{\text{expand}} = 4$	98.88	91.34	1669.35
Expansion	$v_{\text{expand}} = 8$	95.31	95.37	6945.63
Expansion	$v_{\text{expand}} = 16$	99.18	92.63	1383.38

Table 5: Ablation summary. Test and $8\times$ accuracy are measured on MQAR; validation PPL is measured after 100M pretraining tokens.

Related work. We provide the full related-work discussion in section E, covering linear attention and fast weights, state-space and hybrid sequence models, gated linear attention, delta-rule and test-time-learning methods, and efficient long-context attention systems.

6 Limitations

KLA improves the local write coefficient but retains the capacity limits of a fixed-size recurrent state. The exact projection result in theorem 1 holds only when $\eta_t = 1$ and $\varepsilon = 0$; the practical model trades exactness for numerical stability and adaptive write strength. In low-precision arithmetic, very small key norms make the choice of ε important. The longest Palindrome setting also shows a case where GDN slightly outperforms KLA, suggesting that structured reversal may need mechanisms beyond key-norm-normalized residual writes.

7 Conclusion

KLA shows that the delta-rule coefficient determines whether a recurrent write solves its local regression problem. Viewing the write as a constrained projection yields the Kaczmarz coefficient, a

self-normalized replacement for GDN’s learned scalar that becomes an orthogonal projection, exact line-search minimizer, and normalized gradient step in the unrelaxed setting.

Empirically, this coefficient improves 0.4B pretraining PPL by 0.41 over GDN, lowers long-context PPL by 3.7–4.1 across tested lengths, improves MQAR $8\times$ extrapolation by 7.03 points, reaches 100% on S-NIAH, and preserves GDN-like prefill while improving 32K decode throughput by $2.1\times$. The ablations identify key-norm normalization as necessary for stable pretraining, while Palindrome remains a boundary case for normalized residual writes. Future work should test larger state layouts, diagonal or expanded gating, and larger-scale pretraining regimes.

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023. URL <https://arxiv.org/abs/2305.13245>.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate, 2016. URL <https://arxiv.org/abs/1409.0473>.
- [3] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xlstm: Extended long short-term memory, 2024. URL <https://arxiv.org/abs/2405.04517>.
- [4] William Brandon, Aniruddha Nrusimha, Kevin Qian, Zachary Ankner, Tian Jin, Zhiye Song, and Jonathan Ragan-Kelley. Striped attention: Faster ring attention for causal transformers, 2023. URL <https://arxiv.org/abs/2311.09431>.
- [5] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers, 2022. URL <https://arxiv.org/abs/2009.14794>.
- [6] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023. URL <https://arxiv.org/abs/2307.08691>.
- [7] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality, 2024. URL <https://arxiv.org/abs/2405.21060>.
- [8] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022. URL <https://arxiv.org/abs/2205.14135>.
- [9] Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, Guillaume Desjardins, Arnaud Doucet, David Budden, Yee Whye Teh, Razvan Pascanu, Nando De Freitas, and Caglar Gulcehre. Griffin: Mixing gated linear recurrences with local attention for efficient language models, 2024. URL <https://arxiv.org/abs/2402.19427>.
- [10] Daniel Y. Fu, Tri Dao, Khaled K. Saab, Armin W. Thomas, Atri Rudra, and Christopher Ré. Hungry hungry hippos: Towards language modeling with state space models, 2023. URL <https://arxiv.org/abs/2212.14052>.
- [11] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=tEYskw1VY2>.
- [12] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces, 2022. URL <https://arxiv.org/abs/2111.00396>.
- [13] Han Guo, Songlin Yang, Tarushii Goel, Eric P. Xing, Tri Dao, and Yoon Kim. Log-linear attention, 2026. URL <https://arxiv.org/abs/2506.04761>.

- [14] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022. URL <https://arxiv.org/abs/2203.15556>.
- [15] Stefan Kaczmarz. Angenäherte auflösung von systemen linearer gleichungen. *Bulletin International de l'Académie Polonaise des Sciences et des Lettres*, pages 355–357, 1937.
- [16] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 13–18 Jul 2020. URL <https://proceedings.mlr.press/v119/katharopoulos20a.html>.
- [17] Tobias Katsch. Gateloop: Fully data-controlled linear recurrence for sequence modeling, 2024. URL <https://arxiv.org/abs/2311.01927>.
- [18] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer, 2020. URL <https://arxiv.org/abs/2001.04451>.
- [19] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avashalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. Jamba: A hybrid transformer-mamba language model, 2024. URL <https://arxiv.org/abs/2403.19887>.
- [20] Zhixuan Lin, Evgenii Nikishin, Xu Owen He, and Aaron Courville. Forgetting transformer: Softmax attention with a forget gate, 2025. URL <https://arxiv.org/abs/2503.02130>.
- [21] Bo Liu, Rui Wang, Lemeng Wu, Yihao Feng, Peter Stone, and Qiang Liu. Longhorn: State space models are amortized online learners, 2024. URL <https://arxiv.org/abs/2407.14207>.
- [22] Hao Liu and Pieter Abbeel. Blockwise parallel transformer for large context models, 2023. URL <https://arxiv.org/abs/2305.19370>.
- [23] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Jiaju Lin, Przemysław Kazienko, Jan Kocon, Jiaming Kong, Bartłomiej Koptyra, Hayden Lau, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Bolun Wang, Johan S. Wind, Stanisław Wozniak, Ruichong Zhang, Zhenyuan Zhang, Qihang Zhao, Peng Zhou, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. RwkV: Reinventing rnns for the transformer era, 2023. URL <https://arxiv.org/abs/2305.13048>.
- [24] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation, 2022. URL <https://arxiv.org/abs/2108.12409>.
- [25] Zhen Qin, Weigao Sun, Dong Li, Xuyang Shen, Weixuan Sun, and Yiran Zhong. Various lengths, constant speed: Efficient language modeling with lightning attention, 2024. URL <https://arxiv.org/abs/2405.17381>.
- [26] Zhen Qin, Songlin Yang, Weixuan Sun, Xuyang Shen, Dong Li, Weigao Sun, and Yiran Zhong. HGRN2: Gated linear RNNs with state expansion. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=y6SqBJfCSk>.
- [27] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9355–9366. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/schlag21a.html>.
- [28] Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992. doi: 10.1162/neco.1992.4.1.131.

- [29] Uri Shaham, Elad Segal, Maor Ivgi, Avia Efrat, Ori Yoran, Adi Haviv, Ankit Gupta, Wenhan Xiong, Mor Geva, Jonathan Berant, and Omer Levy. Scrolls: Standardized comparison over long language sequences, 2022. URL <https://arxiv.org/abs/2201.03533>.
- [30] Noam Shazeer. Fast transformer decoding: One write-head is all you need, 2019. URL <https://arxiv.org/abs/1911.02150>.
- [31] Julien Siems, Timur Carstensen, Arber Zela, Frank Hutter, Massimiliano Pontil, and Riccardo Grazi. Deltaproduct: Improving state-tracking in linear rnns via householder products, 2025. URL <https://arxiv.org/abs/2502.10297>.
- [32] Jimmy T.H. Smith, Andrew Warrington, and Scott Linderman. Simplified state space layers for sequence modeling. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=Ai8Hw3AXqks>.
- [33] Daria Soboleva, Faisal Al-Khateeb, Robert Myers, Jacob R Steeves, Joel Hestness, and Nolan Dey. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama. <https://cerebras.ai/blog/slimpajama-a-627b-token-cleaned-and-deduplicated-version-of-redpajama>, 2023. URL <https://huggingface.co/datasets/cerebras/SlimPajama-627B>.
- [34] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): Rnns with expressive hidden states, 2025. URL <https://arxiv.org/abs/2407.04620>.
- [35] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models, 2023. URL <https://arxiv.org/abs/2307.08621>.
- [36] Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, Wentao Li, Enzhe Lu, Weizhou Liu, Yanru Chen, Weixin Xu, Longhui Yu, Yejie Wang, Yu Fan, Longguang Zhong, Enming Yuan, Dehao Zhang, Yizhi Zhang, T. Y. Liu, Haiming Wang, Shengjun Fang, Weiran He, Shaowei Liu, Yiwei Li, Jianlin Su, Jiezhong Qiu, Bo Pang, Junjie Yan, Zhejun Jiang, Weixiao Huang, Bohong Yin, Jiacheng You, Chu Wei, Zhengtao Wang, Chao Hong, Yutian Chen, Guanduo Chen, Yucheng Wang, Huabin Zheng, Feng Wang, Yibo Liu, Mengnan Dong, Zheng Zhang, Siyuan Pan, Wenhao Wu, Yuhao Wu, Longyu Guan, Jiawen Tao, Guohong Fu, Xinran Xu, Yuzhi Wang, Guokun Lai, Yuxin Wu, Xinyu Zhou, Zhilin Yang, and Yulun Du. Kimi linear: An expressive, efficient attention architecture, 2025. URL <https://arxiv.org/abs/2510.26692>.
- [37] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023. URL <https://arxiv.org/abs/2302.13971>.
- [38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [39] Roger Waleffe, Wonmin Byeon, Duncan Riach, Brandon Norick, Vijay Korthikanti, Tri Dao, Albert Gu, Ali Hatamizadeh, Sudhakar Singh, Deepak Narayanan, Garvit Kulshreshtha, Vartika Singh, Jared Casper, Jan Kautz, Mohammad Shoeybi, and Bryan Catanzaro. An empirical study of mamba-based language models, 2024. URL <https://arxiv.org/abs/2406.07887>.
- [40] Ke Alexander Wang, Jiaxin Shi, and Emily B. Fox. Test-time regression: a unifying framework for designing sequence models with associative memory, 2025. URL <https://arxiv.org/abs/2501.12352>.

- [41] Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity, 2020. URL <https://arxiv.org/abs/2006.04768>.
- [42] Songlin Yang and Yu Zhang. Fla: A triton-based library for hardware-efficient implementations of linear attention mechanism, January 2024. URL <https://github.com/fla-org/flash-linear-attention>.
- [43] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 56501–56523. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/yang24ab.html>.
- [44] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=y8Rm4VNRPH>.
- [45] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=r8H7xhYPwz>.
- [46] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences, 2021. URL <https://arxiv.org/abs/2007.14062>.
- [47] Shuangfei Zhai, Walter Talbott, Nitish Srivastava, Chen Huang, Hanlin Goh, Ruixiang Zhang, and Josh Susskind. An attention free transformer, 2021. URL <https://arxiv.org/abs/2105.14103>.
- [48] Tianyuan Zhang, Sai Bi, Yicong Hong, Kai Zhang, Fujun Luan, Songlin Yang, Kalyan Sunkavalli, William T. Freeman, and Hao Tan. Test-time training done right, 2025. URL <https://arxiv.org/abs/2505.23884>.
- [49] Yu Zhang, Songlin Yang, Ruijie Zhu, Yue Zhang, Leyang Cui, Yiqiao Wang, Bolun Wang, Freda Shi, Bailin Wang, Wei Bi, Peng Zhou, and Guohong Fu. Gated slot attention for efficient linear-time sequence modeling, 2024. URL <https://arxiv.org/abs/2409.07146>.

A Proofs for the Tokenwise Theory

A.1 Proof of Theorem 1

Let $\tilde{S} = \tilde{S}_{t-1}$, $e = e_t$, and $k = k_t$.

Part (ii): exact line search. Consider the one-parameter family $S(\tau) = \tilde{S} + \tau k e^\top$. Substituting into the per-token loss:

$$\begin{aligned}\mathcal{L}_t(S(\tau)) &= \frac{1}{2} \|S(\tau)^\top k - v_t\|_2^2 = \frac{1}{2} \|\tilde{S}^\top k + \tau e \|k\|_2^2 - v_t\|_2^2 \\ &= \frac{1}{2} (-e + \tau \|k\|_2^2 e)^2 = \frac{1}{2} (1 - \tau \|k\|_2^2)^2 \|e\|_2^2.\end{aligned}\tag{10}$$

Setting the derivative to zero: $\tau^* = 1/\|k\|_2^2$.

Constraint satisfaction. Substituting τ^* :

$$S_t^\top k = \tilde{S}^\top k + e \frac{k^\top k}{\|k\|_2^2} = \tilde{S}^\top k + e = v_t.\tag{11}$$

Hence $S_t \in \mathcal{A}_t$.

Part (i): orthogonal projection. The tangent space of $\mathcal{A}_t$ is $T_{\mathcal{A}_t} = \{H \in \mathbb{R}^{d_k \times d_v} : H^\top k = 0\}$. The correction $\Delta = S_t - \tilde{S} = ke^\top / \|k\|_2^2$. For any $H \in T_{\mathcal{A}_t}$:

$$\langle \Delta, H \rangle_F = \text{tr} \left(\frac{1}{\|k\|_2^2} ke^\top H^\top \right) = \frac{1}{\|k\|_2^2} e^\top (H^\top k) = 0, \quad (12)$$

since $H^\top k = 0$. Therefore $\Delta \perp T_{\mathcal{A}_t}$, confirming that S_t is the orthogonal projection of $\tilde{S}$ onto $\mathcal{A}_t$.

Part (iii): normalized SGD. The gradient of $\mathcal{L}_t$ at $\tilde{S}$ is $\nabla_S \mathcal{L}_t|_{S=\tilde{S}} = k(\tilde{S}^\top k - v_t)^\top = -ke^\top$. Therefore:

$$S_t = \tilde{S} - \frac{1}{\|k\|_2^2} \nabla_S \mathcal{L}_t|_{S=\tilde{S}},$$

which is normalized gradient descent with coefficient $1/\|k\|_2^2$.

Remark 2. By the projection theorem for Hilbert spaces, S_t is the unique element of $\mathcal{A}_t$ satisfying $\|S_t - \tilde{S}\|_F \leq \|S - \tilde{S}\|_F$ for all $S \in \mathcal{A}_t$, with equality only when $S = S_t$. The Kaczmarz update makes the minimum Frobenius-norm change to the state.

A.2 Alternative derivation via Lagrange multipliers

The Lagrangian of (1) is $\mathcal{J}(S, \lambda) = \frac{1}{2} \|S - \tilde{S}\|_F^2 + \lambda^\top (S^\top k - v_t)$ with $\lambda \in \mathbb{R}^{d_v}$. Setting $\partial \mathcal{J} / \partial S = 0$:

$$S - \tilde{S} + k\lambda^\top = 0 \implies S = \tilde{S} - k\lambda^\top.$$

Substituting into $S^\top k = v_t$:

$$(\tilde{S} - k\lambda^\top)^\top k = v_t \implies \lambda = \frac{\tilde{S}^\top k - v_t}{\|k\|_2^2} = -\frac{e}{\|k\|_2^2}. \quad (13)$$

Therefore $S_t = \tilde{S} + ke^\top / \|k\|_2^2$, recovering the Kaczmarz update. $\square$

A.3 Proof of Proposition 1

The proximal problem is:

$$\min_S \frac{1}{2} \|S - \tilde{S}\|_F^2 + \frac{\mu_t}{2} \|S^\top k - v_t\|_2^2.$$

Setting the gradient to zero: $S - \tilde{S} + \mu_t k(S^\top k - v_t)^\top = 0$. Substituting $S = \tilde{S} + \tau ke^\top$:

$$\begin{aligned} \tau ke^\top + \mu_t k(\tau \|k\|_2^2 e - e)^\top &= 0 \\ k e^\top (\tau + \mu_t \tau \|k\|_2^2 - \mu_t) &= 0. \end{aligned} \quad (14)$$

Solving: $\tau = \mu_t / (1 + \mu_t \|k\|_2^2)$. Substituting $\mu_t = \eta_t / ((1 - \eta_t) \|k\|_2^2 + \varepsilon)$:

$$\tau = \frac{\eta_t}{(1 - \eta_t) \|k\|_2^2 + \varepsilon + \eta_t \|k\|_2^2} = \frac{\eta_t}{\|k\|_2^2 + \varepsilon}, \quad (15)$$

which is exactly the KLA coefficient. $\square$

A.4 Proof of Proposition 2

Starting from (3):

$$S_t^\top k = \tilde{S}^\top k + \frac{\eta_t}{\|k\|_2^2 + \varepsilon} e (k^\top k) = \tilde{S}^\top k + \eta_t \frac{\|k\|_2^2}{\|k\|_2^2 + \varepsilon} e. \quad (16)$$

Therefore:

$$e_t^+ = v_t - S_t^\top k = e - \eta_t \frac{\|k\|_2^2}{\|k\|_2^2 + \varepsilon} e = \left(1 - \eta_t \frac{\|k\|_2^2}{\|k\|_2^2 + \varepsilon}\right) e. \quad (17)$$

For $\eta_t \in (0, 1]$ and $\varepsilon \geq 0$, the factor lies in $[0, 1]$, so $\|e_t^+\|_2 < \|e_t\|_2$. Squaring both sides gives $\mathcal{L}_t(S_t) \leq \mathcal{L}_t(\tilde{S}_{t-1})$. $\square$

Method	Recurrent form	Chunkwise-parallel form
Softmax attention	$\sum_{j \leq t} \exp(q_t^\top k_j) v_j$	$(\exp(QK^\top) \odot M)V$
Linear attention	$\sum_{j \leq t} (q_t^\top k_j) v_j$	$(QK^\top \odot M)V$
RetNet	$\sum_{j \leq t} q_t^\top (\prod_{s=j+1}^t \alpha_s) k_j v_j$	$(QK^\top \odot A \odot M)V$
Mamba2	$\sum_{j \leq t} q_t^\top (\prod_{s=j+1}^t \alpha_s) k_j v_j$	$(QK^\top \odot A \odot M)V$
GLA	$\sum_{j \leq t} q_t^\top (\prod_{s=j+1}^t \text{Diag}(\alpha_s)) k_j v_j$	$((Q \odot \Gamma)(K/\Gamma)^\top \odot M)V$
DeltaNet	$\sum_{j \leq t} q_t^\top (\prod_{s=j+1}^t (I - \beta_s k_s k_s^\top)) k_j v_j$	$(QK^\top \odot M) \mathcal{T}_\beta(V)$
GDN	$\sum_{j < t} q_t^\top (\prod_{s=j+1}^t \alpha_s (I - \beta_s k_s k_s^\top)) k_j v_j$	$(QK^\top \odot A \odot M) \mathcal{T}_\beta(V)$
KLA	$\sum_{j \leq t} q_t^\top (\prod_{s=j+1}^t \alpha_s (I - \beta_s k_s k_s^\top)) k_j v_j$	$(QK^\top \odot A \odot M) \mathcal{T}_{\text{kacz}}(V)$

Table 6: Recurrent and chunkwise-parallel forms. M is the causal mask; A and Γ are accumulated decay factors. KLA has the same recurrent form as GDN; only the coefficient $\beta_t = \eta_t / (\|k_t\|_2^2 + \varepsilon)$ differs.

A.5 Decay-agnostic remark

Let D_t be any linear operator and $\tilde{S} = D_t S_{t-1}$. The projection problem $\min_{S \in \mathcal{A}_t} \|S - \tilde{S}\|_F$ depends only on the base point $\tilde{S}$ and the constraint $\mathcal{A}_t$. The derivation of theorem 1 uses only $e = v_t - \tilde{S}^\top k$ and $\|k\|_2^2$, making no assumption about how $\tilde{S}$ was produced from S_{t-1} . The Kaczmarz coefficient applies regardless of whether $D_t = \alpha_t I$, $D_t = \text{Diag}(\alpha_t)$, or any other linear decay.

B Derivations for Chunkwise Parallelism

Table 6 summarizes the recurrent and chunkwise-parallel forms referenced in section 4.

We first recall the chunk-level recurrence. Consider a chunk of length C with local indices $i = 1, \dots, C$. The incoming transposed state is $R_0 = S_0^\top \in \mathbb{R}^{d_v \times d_k}$. From eq. (6):

$$\begin{aligned}
R_i &= \underbrace{\left(\prod_{r=1}^i \alpha_r (I - \beta_r k_r k_r^\top) \right)}_{:= P_i} \cdot R_0 + \underbrace{\sum_{j=1}^i \left(\prod_{r=j+1}^i \alpha_r (I - \beta_r k_r k_r^\top) \right) \cdot \beta_j v_j k_j^\top}_{:= H_i} \\
&= P_i \cdot R_0 + H_i
\end{aligned}$$

where $\beta_i = \eta_i / (\|k_i\|_2^2 + \varepsilon)$. Our goal is to transform P_i and H_i into forms suitable for parallel computation.

Define the cumulative decay $\gamma_i = \prod_{r=1}^i \alpha_r$ ($\gamma_0 = 1$) and the cumulative decay from position j to position i :

$$\gamma_{j \rightarrow i} = \prod_{r=j}^i \alpha_r = \frac{\gamma_i}{\gamma_{j-1}}.$$

We show that P_i , a product of scalar-decayed Householder-like matrices, admits a WY representation.

Proposition 5 (WY representation of P_i). *The matrix P_i can be expressed as:*

$$P_i = \gamma_i I - \sum_{r=1}^i \gamma_{r \rightarrow i} k_r w_r^\top \quad (18)$$

where the auxiliary vector $w_r \in \mathbb{R}^{d_k}$ is computed via the recurrence:

$$w_i = \beta_i \left(\gamma_{i \rightarrow i} k_i - \sum_{r=1}^{i-1} w_r (k_r^\top \gamma_{r \rightarrow i} k_i) \right). \quad (19)$$

Proof. We proceed by induction on i .

Inductive step. Assume $P_{i-1} = \gamma_{i-1}I - \sum_{r=1}^{i-1} \gamma_{r \rightarrow i-1} k_r w_r^\top$. We derive:

$$\begin{aligned}
P_i &= \alpha_i(I - \beta_i k_i k_i^\top) P_{i-1} \\
&= \alpha_i(I - \beta_i k_i k_i^\top) \left(\gamma_{i-1}I - \sum_{r=1}^{i-1} \gamma_{r \rightarrow i-1} k_r w_r^\top \right) \\
&= \alpha_i \left(\gamma_{i-1}I - \sum_{r=1}^{i-1} \gamma_{r \rightarrow i-1} k_r w_r^\top \right) - \alpha_i \beta_i k_i k_i^\top \left(\gamma_{i-1}I - \sum_{r=1}^{i-1} \gamma_{r \rightarrow i-1} k_r w_r^\top \right) \\
&= \gamma_i I - \sum_{r=1}^{i-1} \gamma_{r \rightarrow i} k_r w_r^\top - \beta_i k_i \left(\gamma_{i \rightarrow i} k_i - \sum_{r=1}^{i-1} w_r (k_r^\top \gamma_{r \rightarrow i} k_i) \right)^\top \\
&= \gamma_i I - \sum_{r=1}^{i-1} \gamma_{r \rightarrow i} k_r w_r^\top - k_i \underbrace{\beta_i \left(\gamma_{i \rightarrow i} k_i - \sum_{r=1}^{i-1} w_r (k_r^\top \gamma_{r \rightarrow i} k_i) \right)^\top}_{w_i^\top} \\
&= \gamma_i I - \sum_{r=1}^{i-1} \gamma_{r \rightarrow i} k_r w_r^\top - k_i w_i^\top \\
&= \gamma_i I - \sum_{r=1}^i \gamma_{r \rightarrow i} k_r w_r^\top
\end{aligned}$$

where in the fourth line we used $\alpha_i \gamma_{i-1} = \gamma_i$ and $\alpha_i \gamma_{r \rightarrow i-1} = \gamma_{r \rightarrow i}$, and in the fifth line we factored out k_i and identified the underbraced expression as (19). $\square$

Similarly, H_i can be expressed in a parallelizable form.

Proposition 6 (WY representation of H_i). *The matrix H_i can be expressed as:*

$$H_i = \sum_{r=1}^i \gamma_{r \rightarrow i} k_r u_r^\top \quad (20)$$

where the auxiliary vector $u_r \in \mathbb{R}^{d_v}$ is computed via the recurrence:

$$u_i = \beta_i \left(v_i - \sum_{r=1}^{i-1} u_r (k_r^\top \gamma_{r \rightarrow i} k_i) \right). \quad (21)$$

Proof. We again use induction on i .

Inductive step. Assume $H_{i-1} = \sum_{r=1}^{i-1} \gamma_{r \rightarrow i-1} k_r u_r^\top$.

$$\begin{aligned}
H_i &= \alpha_i(I - \beta_i k_i k_i^\top) H_{i-1} + \beta_i v_i k_i^\top \\
&= \alpha_i(I - \beta_i k_i k_i^\top) \left(\sum_{r=1}^{i-1} \gamma_{r \rightarrow i-1} k_r u_r^\top \right) + \beta_i v_i k_i^\top \\
&= \alpha_i \sum_{r=1}^{i-1} \gamma_{r \rightarrow i-1} k_r u_r^\top - \alpha_i \beta_i k_i k_i^\top \sum_{r=1}^{i-1} \gamma_{r \rightarrow i-1} k_r u_r^\top + \beta_i v_i k_i^\top \\
&= \sum_{r=1}^{i-1} \gamma_{r \rightarrow i} k_r u_r^\top - \beta_i k_i \left(\sum_{r=1}^{i-1} (k_r^\top \gamma_{r \rightarrow i} k_i) u_r \right)^\top + \beta_i v_i k_i^\top \\
&= \sum_{r=1}^{i-1} \gamma_{r \rightarrow i} k_r u_r^\top + k_i \underbrace{\beta_i \left(v_i - \sum_{r=1}^{i-1} u_r (k_r^\top \gamma_{r \rightarrow i} k_i) \right)^\top}_{u_i^\top}
\end{aligned}$$

$$\begin{aligned}
&= \sum_{r=1}^{i-1} \gamma_{r \rightarrow i} k_r u_r^\top + k_i u_i^\top \\
&= \sum_{r=1}^i \gamma_{r \rightarrow i} k_r u_r^\top
\end{aligned}$$

where in the fourth line we used $\alpha_i \gamma_{r \rightarrow i-1} = \gamma_{r \rightarrow i}$ and rearranged transposes, and in the fifth line we identified the underbraced expression as (21). $\square$

B.1 Combined WY representation

Combining Propositions 5 and 6, the state at position i is:

$$R_i = P_i \cdot R_0 + H_i = \gamma_i R_0 - \sum_{r=1}^i \gamma_{r \rightarrow i} k_r (R_0^\top w_r)^\top + \sum_{r=1}^i \gamma_{r \rightarrow i} k_r u_r^\top = \gamma_i R_0 + \sum_{r=1}^i \gamma_{r \rightarrow i} k_r \hat{u}_r^\top, \quad (22)$$

where $\hat{u}_r = u_r - R_0^\top w_r$ combines both auxiliary vectors. In practice, it is more efficient to absorb the R_0 terms directly into the u recurrence. Substituting $\gamma_{r \rightarrow i} = \gamma_i / \gamma_r$ when $\gamma_r > 0$ and defining $\bar{u}_r := \hat{u}_r$ gives the representation underlying the chunkwise system in proposition 3.

B.2 Matrix form (Proposition 3 proof)

Stack all vectors row-wise:

$$K = \begin{bmatrix} k_1^\top \\ \vdots \\ k_C^\top \end{bmatrix} \in \mathbb{R}^{C \times d_k}, \quad V = \begin{bmatrix} v_1^\top \\ \vdots \\ v_C^\top \end{bmatrix} \in \mathbb{R}^{C \times d_v}, \quad U = \begin{bmatrix} u_1^\top \\ \vdots \\ u_C^\top \end{bmatrix} \in \mathbb{R}^{C \times d_u}.$$

The Gram matrix is $G = K K^\top \in \mathbb{R}^{C \times C}$ with $G_{ij} = k_i^\top k_j$. For the row corresponding to position i , the incoming-state prediction is $(\gamma_i R_0 k_i)^\top = (D_\gamma K R_0^\top)_i$, where $D_\gamma = \text{Diag}(\gamma_1, \dots, \gamma_C)$.

Define the strict lower-triangular matrix:

$$L_{ij} = \begin{cases} \beta_i \frac{\gamma_i}{\gamma_j} G_{ij}, & j < i, \\ 0, & j \geq i. \end{cases}$$

Using the causal decay matrix $A_{ij}^- = \gamma_i / \gamma_j$ for $j < i$ (zero otherwise), we have $L = B(A^- \odot G)$ with $B = \text{Diag}(\beta_1, \dots, \beta_C)$.

The u -recurrence (21) (augmented with the R_0 terms from (22)) in matrix form reads:

$$U = B(V - D_\gamma K R_0^\top) - LU,$$

which rearranges to:

$$(I + L)U = B(V - D_\gamma K R_0^\top).$$

Since $R_0 = S_0^\top$ and $L = B(A^- \odot K K^\top)$, this is eq. (7). The matrix $I + L$ is unit lower-triangular (its diagonal entries are all 1, since L is strictly lower-triangular), so it is always nonsingular and is solved by forward substitution. $\square$

B.3 Chunk outputs and outgoing state (Proposition 4 proof)

Token outputs. The output at position i is $o_i = R_i q_i$. Substituting the combined WY representation (22):

$$o_i = \gamma_i R_0 q_i + \sum_{j=1}^i \frac{\gamma_i}{\gamma_j} u_j (k_j^\top q_i).$$

Stacking all $i = 1, \dots, C$:

$$O = D_\gamma Q S_0 + (A \odot Q K^\top) U,$$

where $A_{ij} = \gamma_i / \gamma_j$ for $j \leq i$ (the full lower-triangular version including the diagonal). This is eq. (8). The first term is the readout from the incoming state after decay; the second is the contribution of updates within the chunk.

Outgoing state. Setting $i = C$ in the combined representation and transposing back:

$$\begin{aligned} S_{\text{out}} &= R_C^\top = \left(\gamma_C R_0 + \sum_{j=1}^C \frac{\gamma_C}{\gamma_j} u_j k_j^\top \right)^\top \\ &= \gamma_C S_0 + \sum_{j=1}^C \frac{\gamma_C}{\gamma_j} k_j u_j^\top \\ &= \gamma_C S_0 + K^\top \text{Diag}\left(\frac{\gamma_C}{\gamma_1}, \dots, \frac{\gamma_C}{\gamma_C}\right) U, \end{aligned}$$

which is eq. (9). □

B.4 Why coefficient substitution suffices

The entire derivation uses β_i only through the diagonal matrix $B = \text{Diag}(\beta_1, \dots, \beta_C)$ and the recurrences (19)–(21). No structural property of the GDN coefficient—such as the absence of $\|k_i\|_2$ in the denominator—is invoked at any step. Replacing $\beta_i = \eta_i$ with $\beta_i = \eta_i / (\|k_i\|_2^2 + \varepsilon)$ therefore produces the complete KLA chunkwise algorithm by substituting a different diagonal matrix B into the same GDN kernel. No new CUDA kernel or parallel reduction is required.

B.5 Extension to diagonal gating

When the decay operator is diagonal, $D_t = \text{Diag}(\alpha_t)$ (as in GLA and KDA [36]), the scalar γ_i becomes a vector γ_i and all decay products become elementwise. The WY propositions generalize by replacing $\gamma_{r \rightarrow i}$ with $\text{Diag}(\gamma_{r \rightarrow i})$ wherever it appears. The local Kaczmarz coefficient still applies unchanged; only the transition algebra inside the chunk changes. The chunkwise derivation for diagonal-gated KLA follows the KDA template with B substituted accordingly.

C Experimental Details

Architecture. All models use 24 layers, hidden size 1024, 8 attention heads, head dimension 128, and FFN expansion $4 \times$ (0.4B parameters total).

Pretraining hyperparameters. AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1, gradient clipping 1.0, cosine learning rate schedule with linear warmup over 2% of training steps. Stabilizer $\varepsilon = 10^{-6}$.

Compute resources. All reported experiments used 4 NVIDIA A100 80GB GPUs. Each 1B-token pretraining run took approximately 4 hours. We did not run multi-seed pretraining because of limited compute.

Synthetic tasks. All tasks follow the GDN [45] configurations with seed 42. MQAR: 2-layer model, 10K steps, trained at length 256. S-NIAH: 6K steps. Palindrome: 20K steps. Stack: 20K steps.

D Ablation Study Experimental Setup

For the ablations in Section 5.5, each architectural variant uses the same task pipeline and random seed (42).

MQAR Evaluation Protocol. The Multi-Query Associative Recall (MQAR) datasets use sequence length 256, key length 1, and 32 key-value pairs. Keys are sampled from the first half of the vocabulary, and values from the second half. The split contains 20,000 training, 2,000 validation, and 2,000 test sequences. All variants train for at most 10,000 steps with batch size 32, learning rate 10^{-3} , and weight decay 0.1. Early stopping uses a patience of 10 evaluations, with evaluation every 200 steps. Length extrapolation is evaluated at test factors of $1 \times$, $2 \times$, $4 \times$, and $8 \times$, prioritizing the $8 \times$ metric for final reporting.

Model	Update mechanism	Gating granularity	Normalization / step size	Theoretical basis
Mamba2 [7]	Additive	Scalar (decay)	None	Continuous-time SSM
GLA [43]	Additive	Diagonal (decay)	None	Fast weights
KDA [36]	Delta rule	Diagonal (decay)	Local ℓ_2 + exponentiation	Heuristic scaling
GDN [45]	Delta rule	Scalar (decay + update)	Learned scalar η_t	Heuristic design
KLA (Ours)	Delta rule	Scalar (decay + update)	Kaczmarz $\eta_t / (\ k_t\ _2^2 + \varepsilon)$	Per-token optimization

Table 7: Positioning of KLA against representative sub-quadratic sequence models. KLA is the only model in the table that derives the key-norm normalization in its delta-rule step from exact per-token loss minimization.

100M-Token Pretraining Protocol. To evaluate general language modeling stability, each variant undergoes pretraining on a 100M-token subset of the SlimPajama training split. We use sequence length 4096 and an optimizer learning rate of 10^{-4} . Validation perplexity is computed over 20 validation runs, each averaging 15 iterations. For high-capacity ablations such as $v_{\text{expand}} \in \{8, 16\}$, the micro-batch size is adjusted downward to avoid out-of-memory errors without changing the global token budget.

Variant Configurations. The ablation baseline uses the default KLA configuration: $\beta_t = \eta_t / (\|k_t\|_2^2 + \varepsilon)$ with dual gating.

- **Ablation 1 (Normalization):** Changes the coefficient β_t . The “Learned Scalar” initializes a trainable parameter at 1.0. “NoNorm” reverts to a standard sigmoid gating $\beta_t = \sigma(b_t)$.
- **Ablation 2 (Sequence Factor):** Multiplies the KLA coefficient by $1/t$, $1/\sqrt{t}$, or $1/\log(t+1)$, where t is the absolute sequence index.
- **Ablation 2 (Gating):** Changes the recurrent decay and update gates. “Single Gate” binds the decay and delta-update to a single projected mechanism.
- **Ablation 3 (State Dimension):** Changes the internal value expansion ratio in the linear-attention projection block.

E Related Work

Positioning. We compare sub-quadratic sequence models by the part of the long-context mechanism they change: the attention approximation, recurrent transition, gate, state layout, or online write rule. Table 7 localizes KLA in this design space. The claim of this paper is intentionally narrow: given the scalar-gated delta-rule template of GDN, the write coefficient should be the closed-form Kaczmarz step size rather than a learned or heuristic scalar. Thus, KLA keeps the GDN state, gates, recurrence, and chunkwise solver, but changes the coefficient that determines whether each residual write solves its local regression problem.

Linear attention, RNNs, and fast weights. Linear attention replaces softmax attention with kernelized inner products and exposes an equivalent recurrent form with a fixed-size associative state [16]. Fast-weight interpretations make the same object explicit: the state is a rapidly updated matrix written by the sequence itself [27, 28]. RetNet, RWKV, and Attention-Free Transformer refine retention or channel mixing, while xLSTM, HGRN2, and GateLoop add extended memory, state expansion, or data-controlled transitions [35, 23, 47, 3, 26, 17]. These models establish the value of fixed-size recurrent memory, but they do not isolate the delta-rule step length as an optimization target. KLA follows the associative-state view, yet it does not introduce a new recurrent cell or a larger state; it isolates the rank-one residual write coefficient and normalizes it by the current key norm.

State-space and hybrid sequence models. State-space models pursue linear-time sequence modeling through structured latent dynamics rather than matrix-valued associative writes. S4, H3, and S5 use structured state transitions or simplified state-space layers [12, 10, 32]. Mamba and Mamba2 scale selective input-dependent dynamics and structured state-space duality [11, 7, 39]. Hybrid systems such as Griffin and Jamba combine gated linear recurrences or Mamba-style layers with local

or full attention to recover attention-like behavior [9, 19]. These works optimize the latent transition or the mixture of recurrent and attention layers. KLA addresses a different object: a matrix-state delta-rule update whose content selectivity comes from projecting along the current key direction.

Gated linear attention and state allocation. Gated linear attention adds data-dependent decay to linear attention while preserving hardware-efficient training [43]. Subsequent variants change how the recurrent state is allocated, decayed, or parallelized. Examples include constant-speed Lightning Attention, slot-based gated state, log-linear attention, and Kimi Linear’s KDA with diagonal gating and a specialized chunkwise solver [25, 49, 13, 36]. This line of work mainly asks how expressive the gate and state layout should be. KLA asks an orthogonal question: for a fixed GDN-style scalar gate and state shape, what coefficient makes the residual write an exact local projection in the unrelaxed limit?

Delta-rule models and test-time learning. Delta-rule models replace additive fast-weight writes with residual-correcting updates. DeltaNet parallelizes these updates over sequence length, and GDN adds scalar decay and update gates; section 2 gives their formulations because they are the direct predecessors of KLA [44, 45]. Related online-learning views include Longhorn’s amortized online learner and DeltaProduct’s Householder-product state-tracking mechanism [21, 31]. Broader test-time-learning methods adapt hidden states or weights during inference through expressive inner learners, regression views, forgetting mechanisms, or test-time-training objectives [34, 40, 20, 48]. KLA is closest to GDN, but the modification is smaller and more specific than these broader online learners: it adds no auxiliary model, no iterative inner loop, and no new state type. Its contribution is to replace the learned scalar step with the Kaczmarz coefficient for the single-constraint regression problem.

Efficient attention and long-context systems. Efficient-attention methods reduce the cost of softmax attention through IO-aware kernels, work partitioning, sparse patterns, low-rank or kernelized approximations, blockwise processing, and ring-style sequence parallelism [8, 6, 18, 46, 41, 5, 22, 4]. Decoding-oriented variants such as multi-query and grouped-query attention reduce key-value cache size or head redundancy while keeping the attention interface [30, 1]. These methods make token-token attention cheaper, sparser, lower-rank, or more parallel. KLA removes the token-token attention matrix during autoregressive generation by using a fixed-size recurrent state, and its Kaczmarz coefficient fits into the existing GDN chunkwise algorithm without changing the kernel structure.